Universidade Federal Rural de Pernambuco
Unidade Acadêmica de Belo Jardim

Algoritmos e Estruturas de Dados

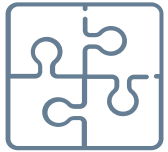
Análise Assintótica

Prof. Waldemar Ferreira **Neto**
Curso: **Engenharia da Computação**
Disciplina: **Algoritmos e Estruturas de Dados**

O Que Vimos na Última Aula?

Diminuir essa distância e participação na aula

É uma atividade difícil mas facilita com a prática e interação



Provas e exercícios

Análise de algoritmos

O que é relevante investigar sobre um algoritmo?

- legibilidade, simplicidade,modularidade, facilidade de manutenção...
- corretude
- **eficiência/desempenho**

Análise de eficiência: o que?

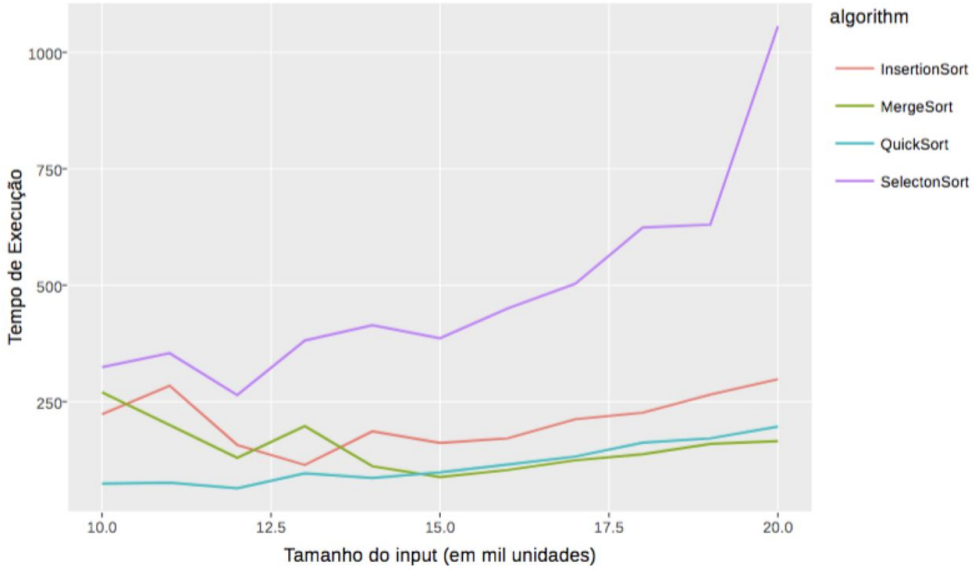
Determinar a quantidade de recursos computacionais que um algoritmo irá consumir.

Recursos computacionais: processamento e memória.

Análise de eficiência: Para que?

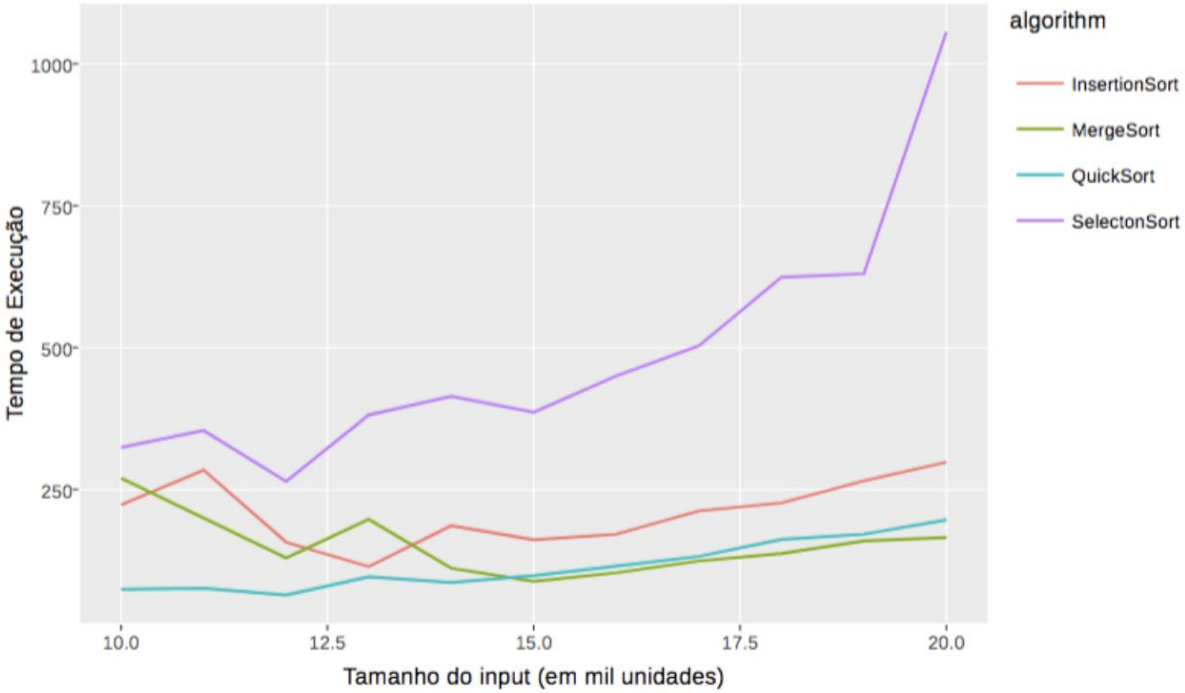
Determinar o comportamento do algoritmo para diferentes tamanhos de entrada.

Comparar diferentes soluções para o mesmo problema.



Análise de eficiência: Como?

Análise Empírica



É precisa,

Mas é cara

Análise de eficiência: Como?

Muitas vezes, não queremos esse nível de precisão.
Nesses casos, precisamos de um método:

- Simples.
- Independente de plataforma.
- Permite generalização para um espectro maior de entrada.
- Análise sem execução.

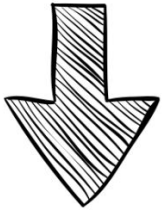
Análise de eficiência: método analítico

```
public static int search(int[] v, int k) {

    for (int i = 0; i < v.length; i++)
        if (v[i] == k)
            return i;

    return -1;

}
```



$$f(n) = c_1 * n + c_2$$

Análise de eficiência: Operações primitivas

O custo de execução das operações primitivas é constante.

- Avaliação de expressões booleanas (i >= 2; i == 2, etc);
- Operações matemáticas (*, -, +, %, etc);
- Retorno de métodos (return x;);
- Atribuição (i = 2);
- Acesso à variáveis e posições arbitrárias de um array (v[i]).

Análise de eficiência: o processo

```
int multiplicaRestoPorParteInteira(int i, int j) {
    int resto = i % j;
    int pInteira = i / j;
    int resultado = resto * pInteira;
    return resultado;
}
```

1. Identificar as primitivas
2. Identificar quantas vezes cada uma das primitivas é executada.
3. Somar o custo total.

Análise de eficiência: o processo

```
int multiplicaRestoPorParteInteira(int i, int j) {
    int resto = i % j;
    int pInteira = i / j;
    int resultado = resto * pInteira;
    return resultado;
}
```

c_1 -> atribuição (resto =)

c_2 -> operação aritmética (i % j)

c_3 -> atribuição (pInteira =)

c_4 -> operação aritmética (i / j)

c_5 -> atribuição (resultado =)

c_6 -> operação aritmética (resto * pInteira)

c_7 -> retorno de método (return resultado)

Análise de eficiência: o processo

```
int multiplicaRestoPorParteInteira(int i, int j) {
    int resto = i % j;
    int pInteira = i / j;
    int resultado = resto * pInteira;
    return resultado;
}
```

1* c_1 -> atribuição (resto =)

1* c_2 -> operação aritmética (i % j)

1* c_3 -> atribuição (pInteira =)

1* c_4 -> operação aritmética (i / j)

1* c_5 -> atribuição (resultado =)

1* c_6 -> operação aritmética (resto * pInteira)

1* c_7 -> retorno de método (return resultado)

Análise de eficiência: o processo

```
int multiplicaRestoPorParteInteira(int i, int j) {
    int resto = i % j;
    int pInteira = i / j;
    int resultado = resto * pInteira;
    return resultado;
}
```

1* c_1 -> atribuição (resto =)

1* c_2 -> operação aritmética (i % j)

1* c_3 -> atribuição (pInteira =)

1* c_4 -> operação aritmética (i / j)

1* c_5 -> atribuição (resultado =)

1* c_6 -> operação aritmética (resto * pInteira)

1* c_7 -> retorno de método (return resultado)

$$f(n) = c_1 + c_2 + c_3 + c_4 + c_5 + c_6 + c_7$$

Análise de eficiência: o processo

```
int multiplicaRestoPorParteInteira(int i, int j) {
    int resto = i % j;
    int pInteira = i / j;
    int resultado = resto * pInteira;
    return resultado;
}
```

f(n) = c1 + c2 + c3 + c4 + c5 + c6 + c7

O tempo de execução é constante. Ou seja, independe do tamanho da entrada.

Outro Exemplo: condicionais

```
double precisaNaFinal(double nota1, double nota2, double nota3) {

    double media = (nota1 + nota2 + nota3) / 3;

    if (media >= 7 || media < 4) {
        return 0;

    } else {
        double mediaFinal = 5;
        double pesoFinal = 0.4;
        double pesoMedia = 0.6;
        double precisa = (mediaFinal - pesoMedia * media) / pesoFinal;

        return precisa;
    }

}
```

Lembrando dos passos:

- identificar as primitivas.
- identificar quantas vezes cada uma das primitivas é executada.
- somar o custo total.

Análise de eficiência: Passo 1

```
double precisaNaFinal(double nota1, double nota2, double nota3) {

    double media = (nota1 + nota2 + nota3) / 3;

    if (media >= 7 || media < 4) {
        return 0;

    } else {
        double mediaFinal = 5;
        double pesoFinal = 0.4;
        double pesoMedia = 0.6;
        double precisa = (mediaFinal - pesoMedia * media) / pesoFinal;

        return precisa;
    }

}
```

- c1...c13.

Análise de eficiência: Passo 1

```
double precisaNaFinal(double nota1, double nota2, double nota3) {

    double media = (nota1 + nota2 + nota3) / 3;

    if (media >= 7 || media < 4) {
        return 0;

    } else {
        double mediaFinal = 5;
        double pesoFinal = 0.4;
        double pesoMedia = 0.6;
        double precisa = (mediaFinal - pesoMedia * media) / pesoFinal;

        return precisa;
    }

}
```

- Estamos interessados no **pior caso**. Portanto, não consideramos a execução da primitiva c5 (return 0).

Análise de eficiência: Passo 1

```
double precisaNaFinal(double nota1, double nota2, double nota3) {

    double media = (nota1 + nota2 + nota3) / 3;

    if (media >= 7 || media < 4) {
        return 0;

    } else {
        double mediaFinal = 5;
        double pesoFinal = 0.4;
        double pesoMedia = 0.6;
        double precisa = (mediaFinal - pesoMedia * media) / pesoFinal;

        return precisa;
    }

}
```

- $f(n) = c1 + c2 + c3 + c4 + c6 + c7 + c8 + c9 + c10 + c11 + c12 + c13$

- O tempo de execução é constante. Ou seja, independe do tamanho da entrada.

Outro Exemplo: loop

```
public static boolean contains(int[] v, int k) {
    for (int i = 0; i < v.length; i++)
        if (v[i] == k)
            return true;
    return false;
}
```

Análise de eficiência: Passo 1

```
public static boolean contains(int[] v, int k) {
    for (int i = 0; i < v.length; i++)
        if (v[i] == k)
            return true;
    return false;
}
```

c_1 -> atribuição (int i = 0)

c_2 -> avaliação de expressão booleana (i < v.length)

c_3 -> operação aritmética (i++)

c_4 -> avaliação de expressão booleana (v[i] == k)

c_5 -> retorno de método (return true)

c_6 -> retorno de método (return false)

Análise de eficiência: Passo 1

```
public static boolean contains(int[] v, int k) {
    for (int i = 0; i < v.length; i++)
        if (v[i] == k)
            return true;
    return false;
}
```

- c1 é executada apenas uma vez.
- c2 é executada (n+1) vezes. Exemplo: se n=5, temos as seguintes verificações: 0 < 5, 1 < 5; 2 < 5, 3 < 5, 4 < 5 e 5 < 5, quando encerra-se o loop. Ou seja, 6 verificações.
- c3 é executada n vezes. Exemplo: se n=5, temos os seguintes incrementos em i: 1, 2, 3, 4 e 5, quando encerra-se o loop.
- c4 é executada n vezes.
- No **pior caso**, c5 não é executada.
- c6 é executada apenas uma vez.

Análise de eficiência: Passo 1

```
public static boolean contains(int[] v, int k) {
    for (int i = 0; i < v.length; i++)
        if (v[i] == k)
            return true;
    return false;
}
```

$f(n) = c_1 + c_2 * (n+1) + c_3 * n + c_4 * n + c_6$

Considerando todas as primitivas com custo c e simplificando a função, temos:

$f(n) = 3 * c * n + 3 * c$ Felipe -> $f(n) = c1 + c2 * (n+1) + c3 * (n * 2) + c4 * n + c5 * n + c6$

O tempo de execução é linear. O valor de f(n) cresce linearmente em função do tamanho da entrada (n).

Desafio

```
public boolean contemDuplicacao(int[] v) {
    for (int i = 0; i < v.length; i++)
        for (int j = i + 1; j < v.length; j++)
            if (v[i] == v[j])
                return true;
    return false;
}
```

f(n) = c1 + c2 * (n + 1) + c3 * (n * 2) + c4 * n + c5 * n + c6 * (n + 1) + c7 * (n * 2) + c8 * n + c9 * n + c10 *n

Resumo

- Estamos interessados na análise de eficiência. Em particular, tempo de processamento.
- Mais recursos computacionais nem sempre resolvem o problema.
- Método empírico é preciso, porém caro para o que queremos.
- Método analítico. Extrair a função que determina do custo de execução de um algoritmo:
  - identificar as primitivas.
  - identificar quantas vezes cada uma das primitivas é executada.
  - somar o custo total.
- Estamos interessados na análise de pior caso.

Obrigado!

Any questions?

Contato:

- wpfn@cin.ufpe.br (provisório)

Classroom:

- **ulqrpjd**